

RESIDUAL-PATHS-AUDIT

Residual /Volumes Paths — Post-Fix Verification Audit

Date: 2026-08-27 **Scope:** every tracked file plus untracked .ashlrcode/genome/** **Method:** read-only.git ls-files | xargs grep -l '/Volumes', then per-occurrence classification and consumer tracing for every path found inside a data file.

Headline

Files containing /Volumes	1,604 tracked (+0 untracked beyond those)
Total occurrences	10,424
scripts/audit-hardcoded-paths.sh verdict	no machine-specific hardcoded paths — exit 0
Would a fresh clone hit any of these at runtime?	NO
INSTRUCTIONAL findings	1 file (_analysis/IMPLEMENTATION-REPORT.md)
LOAD-BEARING findings	0
MISLEADING-TO-AGENTS findings	6 genome files — the genome still asserts the bug is unfixed

The systemic fix is real: _tools/deploy-langs.sh, _tests/playwright.config.js, _tools/translate-pipeline.sh, _tools/review-translations.sh and the translate tooling all derive their roots now. Nothing executable reads a /Volumes literal. What remains is documentation debt, and two pieces of it are actively wrong.

SECTION 1 — MUST FIX

1.1 INSTRUCTIONAL — `_analysis/IMPLEMENTATION-REPORT.md` (30 occurrences)

This file hands a reader a copy-pasteable bash block that `cds` into directories that do not exist on any machine but the author's original macOS box.

`_analysis/` is in the audit's SKIP regex (`scripts/audit-hardcoded-paths.sh:53`), so the audit never sees it.

§8 “Reproduction Commands” (lines 222–250) — 9 executable `/Volumes` paths

```
224: cd <macos-host>/Projects/vasic/milosvasic.ru && jekyll build
225: cd <macos-host>/Projects/vasic/_tests
231: cd <macos-host>/Projects/helix_translate
239: <macos-host>/Projects/vasic/_tools/translate-pipeline.sh \
240:   --in <macos-host>/Projects/vasic/milosvasic.ru/
   _article_src/en/helix-track-core.md \
241:   --out <macos-host>/Projects/vasic/milosvasic.ru/
   _article_src/ru/helix-track-core.md \
245: <macos-host>/Projects/vasic/_tools/render-articles.sh <macos-
host>/Projects/vasic/milosvasic.ru en sr ru
246: <macos-host>/Projects/vasic/_tools/render-articles.sh <macos-
host>/Projects/vasic/vasic.digital en
249: cd <macos-host>/Projects/vasic/milosvasic.ru/downloads/src
&& ./build-pdfs.sh
```

Every one of these targets **does exist in this repo** under a derived root (`_tools/translate-pipeline.sh`, `_tools/render-articles.sh`, `milosvasic.ru/_article_src/`, `vasic.digital/_article_src/` all confirmed present). The commands are correct in substance and wrong only in their anchor — which is the worst failure mode: a reader gets `No such file or directory`, not a hint that the paths are stale. **Verdict: replace the literal prefix with `$(git rev-parse --show-toplevel)` or a relative invocation.**

Line 179 — a factually FALSE statement about current behaviour

```
_tools/translate-pipeline.sh — ... around the engine binary
(HELIX_TRANSLATE_BIN, default <macos-host>/Projects/
helix_translate/build/unified-translator; binary confirmed
present and executable on disk).
```

The real current default is a container shim:

```
_tools/translate-pipeline.sh:55
ENGINE="{HELIX_TRANSLATE_BIN:-$TOOLS/helixtranslate-
container.sh}"
```

The report documents a default that was deliberately removed.
Verdict: stale claim, correct it or mark the section historical.

Remaining 20 occurrences in the same file (lines 6, 7, 11, 21, 22, 30, 44, 48, 54, 62, 66, 72, 114, 158, 193, 199)

Narrative provenance (“Repo: <macos-host>/Projects/vasic/milosvasic.ru”, “File: /Volumes/.../assets/js/i18n.js”). **PROSE** — harmless in isolation, but they are what makes §8 read as authoritative. Recommend a single “paths in this report are from the authoring machine; derive your own root” banner at the top.

1.2 MISLEADING TO AGENTS — the genome asserts the bug is still open (6 files, 10 occurrences)

.ashlrcode/genome/ is consumed by ashlr as **retrieval context**. Six files state, in the present tense, that two scripts still hardcode <macos-host>/Projects/vasic. **They do not.** Verified against the live files:

Claim in genome	Reality
<pre>_tools/deploy-langs.sh:8 — ROOT=<macos-host>/Projects/ vasic"</pre>	Line 8 is now a comment describing the historical bug; line 14 is ROOT="\$(cd -- "\$(dirname -- "\${BASH_SOURCE[0]}")/.." && pwd)" with a fatal cd guard

Claim in genome	Reality
<code>_tests/playwright.config.js</code> hardcodes the same prefix	Line 7 is <code>const REPO = path.resolve(__dirname, '..');</code> — no <code>/Volumes</code> anywhere in the file
“CI works around it by symlinking <code>/Volumes/...</code> -> <code>\$GITHUB_WORKSPACE</code> ”	<code>.github/workflows/ci.yml:164</code> records the symlink bridge is no longer needed

An agent routing off this genome will look for a bug that is fixed, may “re-fix” the comment that documents the fix, or will treat backlog item 11 as outstanding work.

File	Occ	Line(s)	What it says	Verdict
<code>.ashlrcode/genome/knowledge/discoveries.md</code>	4	6, 8, 12, 14	Section heading “ Two scripts hardcode <code><macos-host>/Projects/vasic</code> ” with line-numbered specifics	STALE — highest impact. This is the canonical record every other genome file points at
<code>.ashlrcode/genome/vision/anti-patterns.md</code>	2	31–35	<code>“_tools/deploy-langs.sh:8 sets ROOT=“<macos-host>/Projects/vasic” ... On any checkout not at that path these break”</code>	STALE
<code>.ashlrcode/genome/vision/architecture.md</code>	1	64	<code>“two build scripts hardcode absolute <macos-host>/... paths”</code>	STALE
<code>.ashlrcode/genome/milestones/backlog.md</code>	1	64	Open backlog item “ 11. Repair the two hardcoded <code><ma</code>	STALE — work already done, item should close

File	Occ	Line(s)	What it says	Verdict
.ashlrcode/ genome/ knowledge/ architecture.md	1	70	cos-host>/... paths” Pointer: “Structural hazards (..., hardcoded <macos-host> paths) are in knowledge/ discoveries.md”	STALE pointer (harmless once discoveries.md is corrected)
.ashlrcode/ genome/meta/ maintenance.md	1	59	Routing rule: update discoveries.md “if the hardcoded <macos-host> paths are finally fixed”	CONDITION HAS FIRED — the rule is correct, it just needs executing

These are not INSTRUCTIONAL (nothing says “run this here”) and not LOAD-BEARING (no runtime reads them), but they are the single highest-leverage residual: wrong genome state propagates into agent behaviour by design.

SECTION 2 — VERIFIED HARMLESS

2.1 CORRECTIVE (5 files, 10 occurrences) — desirable, keep

File	Occ	Evidence	Verdict
_tools/deploy- langs.sh	1	L8: # ROOT was hardcoded to "<macos- host>/	CORRECTIVE. Comment-only; the code below derives the root and makes cd fatal

File	Occ	Evidence	Verdict
		Projects/ vasic" - a macOS path. On any other checkout the cd below failed...	
scripts/audit- hardcoded- paths.sh	3	L7, L13 comments; L50 PATTERN= contains the literal / Volumes/	CORRECTIVE + necessarily literal. This IS the detector. Allow- listed in .hardcoded-paths- allow with that exact reason. Audit prints allowed and still exits 0 CORRECTIVE. Comment-only. (<i>Not modified — another actor is editing this file.</i>) Path note (2026-08-27): the file is renamed to a non-active .disabled name under the §11.4.156 compliance decision (<u>../ constitution- adoption/ DECISION-11-4-156- COMPLY.md</u>). The occurrences and the verdict are unchanged — only the path is.
.github/ workflows/ ci.yml → now .github/ workflows/ ci.yml.disabled	3	L44–48 explain the historical symlink; L164 states the bridge “is no longer needed”	
docs/setup- agents-wizard/	2	L331 describes the removed	CORRECTIVE / spec

File	Occ	Evidence	Verdict
OPERATIONAL - SCRIPTS.md		macOS root; L342 documents the detector's alternation	
docs/setup- agents-wizard/ README.md	1	L451 describes test group J and the "18 tracked files once hardcoded one author's / Volumes/... macOS root"	CORRECTIVE

2.2 CORRECTIVE (auto-captured) — .ashlr/code/genome/knowledge/discoveries- auto.md (15 occurrences)

Raw ashlr tool-call JSON auto-appended under
Auto-observations · 2026-08-27. The /Volumes strings sit inside
oldString fields of recorded edits — i.e. they are **records of the
removal**, paired with newString values that derive the root (L171
review-translations.sh, L194 distribute-helixtranslate.sh, L196
translate-pipeline.sh, L296 the audit script's own header).

Verdict: CORRECTIVE, but noisy. It is machine-generated append-
only log, not curated knowledge. Low risk, but it dilutes retrieval — a
consolidation pass would help.

2.3 PROSE — _analysis/content-briefs/*.md (20 files, 32 occurrences)

All occurrences sit under a “**Source provenance:**” heading and cite
sibling repos outside this checkout that were read while authoring the
brief:

HelixQA.md:66 - <macos-host>/Projects/helixqa/README.md (status
banner round 219, ...)

HelixQA.md:67 - <macos-host>/Projects/helixqa/CONSTITUTION.md (inheritance ...)

HelixConstitution.md:62-65 <macos-host>/Projects/constitution/{README.md,Constitution.md,find_constitution.sh,submodules-catalogue.md}

Catalogizer.md:44 - Local working copy: <macos-host>/Projects/catalogizer (has private .env; ...)

HelixCode.md:59 - <macos-host>/Projects/helix_code/README.md (primary; version, architecture, ...)

File	Occ	File	Occ
HelixConstitution.md	5	HelixSkills.md	1
HelixQA.md	4	HelixSpecifier.md	1
Catalogizer.md	2	HelixTrack.md	1
HelixAgent.md	2	HelixTranslate.md	1
HelixCode.md	2	LLMOrchestrator.md	1
HelixLLM.md	2	LLMProvider.md	1
HelixOTA.md	2	LLMsVerifier.md	1
HelixBuilder.md	1	HelixMemory.md	1
HelixCluster.md	1	HelixPlay.md	1
HelixGitpx.md	1	Vasic-Digital-Reusable-Module-Suite.md	1

Verdict: PROSE — harmless. No imperative verb attaches to any of them; they are bibliography entries, not steps. They point at *other projects*, so they cannot be re-anchored to this repo root — the honest fix is a note that provenance was gathered from a sibling-checkout layout, not a path rewrite. **No action required.**

2.4 PROSE (one caveat) — _analysis/ OPENDESIGN-ADOPTION-PLAN.md (1 occurrence)

L13: Every decision here traces to a Helix Constitution mandate (`<macos-host>/Projects/constitution/Constitution.md``).

Verdict: PROSE, minor. The constitution is vendored **in this repo** at submodules/constitution (confirmed present), so this citation sends a reader outside the checkout for a document that is one directory away. Not instructional, but a one-word improvement.

2.5 DATA — `_tests/evidence/**` (1,570 files, 10,326 occurrences)

Skipped by the audit (`_tests/evidence/` in SKIP). Broken down and traced:

`translate-new/` — 537 files (523 `*.review.json` + 14 `batch.log`) — re-anchoring CONFIRMED

Every review JSON carries `"_translated": "<macos-host>/Projects/vasic/_content_<lang>/..."`. **All three live consumers re-anchor before use, and all three derive their own repo root:**

Consumer	Root derivation	Re-anchoring
<code>_tools/translate/matrix.py</code>	<code>pathlib.Path(__file__).resolve().parents[2]</code> , VASIC_ROOT override, fatal if no docs found	<code>anchored()</code> L27–38: <code>re.search(r"(_content[^\", \"]*)", ...)</code> → <code>os.path.join(R...tail)</code> , used at L68
<code>_tools/translate/finalize-review.sh</code>	L38 <code>\${VASIC_ROOT:-\$(cd -- "\$(dirname ...)/.." && pwd)}</code>	inline python L78–86, <code>sar...</code> <code>regex, argv[3] = derived ...</code>
<code>_tools/translate/run-batch.sh</code>	L26, identical derivation	inline python L55–62, <code>sar...</code> <code>regex, argv[2] = derived ...</code>

All three point EVID/BASE at `$REPO/_tests/evidence/translate-new` — never a literal.

Why re-anchoring is what makes them harmless, and why it cannot fake a PASS: each consumer still requires `os.path.isfile()` on the *re-anchored* path. A stale prefix degrades to “file missing” → not-PASS. The design is fail-closed. The code says so explicitly: *“This never fakes a PASS: the re-anchored file still has to actually exist.”* **Confirmed as described. No action.**

translate/review/ — 438 additional *.json with the same _translated field (NOT previously flagged)**

This is a **second, older evidence set** that the brief did not mention. Its paths point at `_article_src`, **not** `_content`:

```
_tests/evidence/translate/review/milosvasic.ru/ar/android-toolkit.json
  "_translated": "<macos-host>/Projects/vasic/milosvasic.ru/_article_src/ar/android-toolkit.md"
```

The re-anchor regex (`_content[^\]*/.*)$` **would not match** `_article_src` — so if anything consumed these, re-anchoring would silently no-op.

It is not consumed. Traced exhaustively: - All three re-anchoring consumers read `translate-new` **only**. - `_tools/review-translations.sh` *writes* into `$ROOT/_tests/evidence/translate/review/$site` (L24, derived root — already fixed) and reads back only `verdict / accuracy / fluency / completeness / script_ok / issues` (L47). **No path field**. - `_tools/translate-fleet.sh`, `translate-all-langs.sh`, `translate-all-defresfr.sh` only write summary logs there.

Verdict: DATA — dead evidence, harmless. Worst case is fail-closed anyway. Flagged here so it is on record that the 537-file figure was an undercount of the `_translated` population (total ≈ 961 JSON across both sets).

ui-l10n/reviews/*.json (6) + l10n-audit/reviews/de.review.json (1)

`"_translated": "<macos-host>/tmp/tmp<random>/tr.md"` — **ephemeral tempdirs** that never survived the run that produced them.

Consumers `_tools/gen/review_ui_all.py` (L40) and `_tools/gen/translate-ui.py` (L124) read only `verdict / accuracy / fluency / completeness / naturalness / terms_verified / terms_required`. **The _translated key is never read.** Verdict: **DATA — harmless.**

harness/{portfolio,export,visual}/golden- {good,bad}.verdict.json (6)

Carry fixture, pdf, pdftotextFile, input (file:///Volumes/...), screenshot, txtFile. **These are outputs, never inputs.** All three harnesses regenerate them from derived roots on every run:

- `_tools/portfolio/self-validate.sh` —
ROOT="\$(cd "\$HERE/../../.." && pwd)", FIX="\$HERE/selfvalidate-fixtures", writes \$OUT/\$name.verdict.json
- `_tests/visual/self-validate.sh` — node "\$HERE/visual-oracle.js"
--input "\$HERE/fixtures/good.html" --out "\$OUT"
- `_tests/export/self-validate.sh` — node "\$HERE/validate-pdf.js"
--pdf "\$FIX/golden-good.pdf" --out "\$OUT"

Verdict: **DATA** — stale build artifacts, overwritten on next run.

All other *.verdict.json (pages/, homepages/, components/, diagrams/, fixes/, final-suite*/, pdf-audit/, ~100 files)

Every code reference to verdict.json in the repo is a **write**:

```
_tests/export/validate-pdf.js:146    path.join(args.out, `${args.name}.verdict.json`)  
_tests/visual/visual-oracle.js:238    path.join(args.out, `${args.name}.verdict.json`)  
_tests/tools/diagram-brand-accent-proof.js:66    path.join(OUT, 'verdict.json')
```

Zero reads anywhere in the repo. Verdict: **DATA** — harmless.

.log / .txt / .tsv evidence (~900 files, the bulk of the 10,326)

Captured stdout from historical runs (`_tests/evidence/translate/*.log` alone accounts for ~7,000 occurrences of `/Volumes/.../_article_src/en/*.md` as echoed filenames). Nothing parses them. Verdict: **DATA** — harmless.

Archived driver scripts under `_tests/evidence/` (23 files) — inert, but worth knowing

These are one-shot drivers left beside their output. They **do** hardcode / Volumes in executable positions — `require()` targets, `REPO=` constants, `cd`:

```
_tests/evidence/ally-audit/run-audit.js:12
require(path.join('<macos-host>/Projects/vasic/_tests/
node_modules/playwright'))
_tests/evidence/ally-fix3/render-axe.js:11      const ROOT =
'<macos-host>/Projects/vasic/milosvasic.ru/_site'
_tests/evidence/fix4docs/retry.sh:4             cd <macos-host>/
Projects/vasic
_tests/evidence/fix4docs/retry.sh:12             bash <macos-
host>/Projects/vasic/_tools/translate/translate-content.sh ...
_tests/evidence/fix-pdf-chrome/
{leak_matrix,review_chrome,translate_chrome}.py  REPO = "<macos-
host>/Projects/vasic"
_tests/evidence/live-v1.3.1/scripts/{axe-live.js,interactive-
live.js,l10n.sh,l10n-vasic.sh,pdf.sh,reach.sh,reach2.sh,scope.sh}
_tests/evidence/live-v1.5.1/{ally/run-axe.mjs,interactive/run-
interactive.mjs}
_tests/evidence/milos-ssr-chrome/playwright-check.js
_tests/evidence/fix-frontend/_run-evidence.mjs
_tests/evidence/ally-fix2/{measure-badges,measure-dialog}.js
```

Verified unreachable. Each basename was searched across every tracked file outside `_tests/evidence/` — **zero references**, for all 15 distinct names checked. No CI step, npm script, test spec or shell glob (`find ... -exec, for ... in _tests/evidence/*.sh`) invokes anything under `_tests/evidence/`.

Verdict: DATA / dead code. They would break if run by hand, but nothing runs them and nothing points at them. Not LOAD-BEARING. Deleting or root-deriving them is optional hygiene, not a fix.

`_tests/evidence/monorepo-health/AUDIT.md` (1) and `closure-final/05-known-open.txt` (many)

Frozen report prose and captured grep output where `/Volumes/...` is the *matched filename column*. Verdict: **DATA** — **harmless**.

SECTION 3 — THE FOUR SPECIFIC QUESTIONS

1. Does any script still READ a /Volumes value out of a data file without re-anchoring it?

No. Every path-valued field in evidence data was enumerated and its consumers traced:

Field	Where	Consumers	Outcome
<code>_translated</code>	<code>translate-new/</code> (523), <code>translate/review/</code> (438), <code>ui-ll0n/</code> (6), <code>ll0n-audit/</code> (1)	<code>matrix.py</code> , <code>finalize-review.sh</code> , <code>run-batch.sh</code>	Re-anchored onto a derived root, with a mandatory <code>isfile()</code> re-check → fail-closed
<code>_translated</code>	<code>ui-ll0n/</code> , <code>ll0n-audit/</code>	<code>review_ui_all.py</code> , <code>translate-ui.py</code>	Never read — only verdict/score fields
<code>_translated</code>	<code>translate/review/</code> (438)	<code>review-translations.sh</code>	Never read — only verdict/score fields
<code>fixture</code> , <code>pdf</code> , <code>pdftotextFile</code> , <code>input</code> , <code>screenshot</code> , <code>txtFile</code>	<code>harness/*goldens</code>	<code>3× self-validate.sh</code>	Never read — regenerated from derived roots
every other <code>*.verdict.json</code> field	all evidence dirs	<code>validate-pdf.js</code> , <code>visual-oracle.js</code> , <code>diagram-brand-accent-proof.js</code>	Write-only — no read site exists

Additionally, `_tools/translate/translate-content.sh`'s `jget` helper (L123) is called only for verdict, accuracy, fluency, completeness, `script_ok`, `untranslated_leftovers` (L235–237) — never a path.

2. Do the genome files mislead an agent using the genome for routing?

Yes — see §1.2. Six files assert in the present tense that `_tools/deploy-langs.sh` and `_tests/playwright.config.js` still hardcode `<macos-host>/Projects/vasic`, and that CI symlinks around it. All three claims are now false. `milestones/backlog.md` item 11 tracks the repair as outstanding work that is already done. This is the most consequential residual in the repository, because the genome is retrieval context by design.

3. Is `_analysis/` referenced by any executable code?

No. Two references exist, neither is a read:

```
_tools/distribute-helixtranslate.sh:7    # Strategy (proven, see
_analysis/CONTAINER-DISTRIBUTION.md):    ← comment
scripts/audit-hardcoded-paths.sh:53      SKIP='^(docs/|_content|
_analysis|_tests/evidence/|...)'         ← exclusion regex
```

`_analysis/` is **purely narrative**. That is precisely what makes §1.1 survivable — and also what let it go unnoticed.

4. Would a fresh clone on a different machine hit ANY of these at runtime?

NO.

- Every executable path in live code derives its root (`BASH_SOURCE/_dirname/_file__`), with `VASIC_ROOT` as an explicit override.
- The only `/Volumes` literal in live code is the audit's own `PATTERN`, which must be literal and is allow-listed for that reason.
- All 10,326 evidence occurrences are in files that are written, not read — or, for `_translated`, are re-anchored with a fail-closed existence check.
- The 23 archived evidence drivers that *would* break are unreachable from any entry point.

A human reading `_analysis/IMPLEMENTATION-REPORT.md` §8 and pasting it will hit No such file or directory — and an agent reading the genome will believe a fixed bug is still open. Both are documentation failures, not runtime failures.

SECTION 4 — IS THE “AUDIT IS CLEAN” CLAIM HONEST?

Yes for what it claims; no for what a reader will infer.

The audit exits 0, and that verdict is **earned**: it proves no *tracked, non-skipped, non-comment* line carries a machine-specific absolute path. Its skips are principled and documented in its own header — “*Prose legitimately quotes real paths; generated evidence is not source*” — and its comment-stripping is deliberate: “*documenting the historical bug is not the bug.*” Both are correct engineering calls. `scripts/audit-hardcoded-paths.sh` sits in `.hardcoded-paths-allow` with the honest reason that it **is** the detector.

Three caveats a reader should carry:

1. **`_analysis/` is skipped on the theory that it is prose. §8 of IMPLEMENTATION-REPORT.md is not prose — it is a runnable command block.** The skip’s justification does not hold for that file. This is the one place where “the audit is clean” is technically true and practically misleading.
2. **`.ashlrcode/` is skipped on the theory that it is generated.** For `discoveries-auto.md` that is right. For the six hand-curated genome files it is wrong — they are authored knowledge, they are wrong, and they feed agent behaviour.
3. **The audit scans `git ls-files` only.** Untracked working files are invisible to it. (Checked for this review: `.ashlrcode/genome/**` holds no untracked `/Volumes` carriers beyond the seven already tracked.)

Recommended follow-up, in priority order — none of it required to make a fresh clone work:

1. Re-anchor `_analysis/IMPLEMENTATION-REPORT.md` §8 and correct the false `HELIX_TRANSLATE_BIN` default at line 179.
 2. Update the six genome files to past tense and close `backlog.md` item 11.
 3. Consider narrowing the audit’s `_analysis` skip to prose-only, or adding a companion check that flags `/Volumes` inside fenced “`bash blocks anywhere in the tree.
-

*Read-only audit. No file other than this report was created or modified. .
github/workflows/ci.yml and scripts/setup-agents-wizard.sh were
read but not touched.*